

CS 301: High-Performance Computing

Lab 03 — Interpolation of Scattered Data on a Structured Grid

Group 1

Submitted by:

Name	Roll Number
Tanuj Kanani	202301474
Chavda Mihirsinh	202301479

Dhirubhai Ambani Institute of Information and
Communication Technology (DA-IICT)

February 24, 2026

Abstract

This report presents a detailed implementation and performance analysis of a serial bilinear interpolation algorithm that maps randomly scattered 2D spatial points onto a structured computational mesh grid. For each particle, the algorithm identifies the four bounding grid nodes and distributes a unit value to them using area-weighted fractions. Performance is benchmarked across five problem configurations (scaling from 0.9 million to 20 million points) on two distinct hardware platforms: a consumer-grade Lab PC (Intel Core i5-12500) and an HPC Cluster (Intel Xeon E5-2620 v3). The analysis reveals that the algorithm is heavily memory-bound, and that single-core clock frequency — not aggregate core count — is the dominant factor in serial execution speed. The Lab PC consistently outperforms the HPC cluster by a factor of approximately $3\times$ across all tested configurations.

Contents

1	Introduction	2
2	Problem Context and Formulation	2
2.1	Problem Description	2
2.2	Algorithm Complexity	2
2.3	Why Memory-Bound?	3
3	Hardware Specifications	3
3.1	Lab PC Hardware Specifications	3
3.2	HPC Cluster Hardware Specifications	3
4	Methodology	3
4.1	Implementation Overview	3
4.2	Bilinear Interpolation Procedure	4
4.3	Key Optimisations Implemented	4
4.4	Accuracy Verification	5
4.5	Problem Configurations	5
5	Results and Performance Analysis	5
5.1	Execution Time Measurements	5
5.2	Throughput Analysis	6
5.3	Performance Plots	6
6	Answers to Lab Questions and Analysis	7
6.1	Q1: Theoretical Time Complexity	7
6.2	Q2: Estimate of Arithmetic Operations per Particle	8
6.3	Q3: Memory Access Analysis and Cache Behaviour	9
6.3.1	Access Patterns	9
6.3.2	Memory Bandwidth Requirement	9
6.3.3	Cache Miss Analysis	9
6.3.4	Why the Flat Array Wins	9
6.4	Q4: Further Optimisations Under Better Hardware Assumptions	9
6.5	Q5: Lab PC vs. HPC Cluster Performance	10
6.5.1	Observed Result	10
6.5.2	Explanation	10
6.5.3	Configuration E Anomaly	10
6.6	Q6: Pictorial Implementation Approach	11
6.6.1	Data Layout Strategy	11
6.6.2	Algorithmic Workflow	11
7	Conclusion	11
A	Appendix: Raw Benchmark Data	12
A.1	Lab PC Raw Times (interpolation_PC.csv)	12
A.2	HPC Cluster Raw Times (interpolation_cluster.csv)	12

1. Introduction

In this laboratory experiment, we investigate the performance characteristics of mapping randomly scattered 2D spatial points onto a structured computational mesh using **Bi-linear Interpolation**. For every scattered particle within the domain, the algorithm identifies the four nearest bounding grid nodes and distributes a localised value ($f_i = 1$) to them based on fractional, area-weighted distances.

We specifically analyse how memory access patterns and hardware-level behaviours dictate overall execution speed. The key benchmarks include:

- The performance penalty of complex data structures (linked lists or spatial bins) versus the spatial locality benefits of a simple, contiguous flat array.
- The utilisation of the CPU's *Hardware Prefetcher* when traversing massive particle datasets stored sequentially.
- The computational impact of floating-point optimisations (replacing division with pre-computed inverse multiplication) and *branchless programming* (using ternary operators to bypass branch-prediction penalties).

By measuring processing time for datasets scaling up to 20 million points across multiple iterations, we determine the most cache-efficient implementation. Benchmarks are recorded on both a consumer-grade Lab PC and an HPC Cluster to observe hardware-specific scaling behaviour.

2. Problem Context and Formulation

2.1 Problem Description

A continuous domain of 1×1 units contains up to $N = 20,000,000$ scattered particles. These must be mapped to an $M \times M$ discrete grid. Each particle updates four separate memory locations on the grid via area-weighted fractions. The formal problem can be stated as follows:

Given N scattered points $\mathcal{S} = \{(x_i, y_i, f_i) \mid i = 1, 2, \dots, N\}$ where $f_i = 1$, interpolate these values onto a structured mesh $\mathcal{G} = \{(X_i, Y_j)\}$ of dimension $N_x \times N_y$ using bilinear interpolation.

2.2 Algorithm Complexity

Time Complexity: $\mathcal{O}(N \cdot \text{Maxiter})$. The algorithm executes a strictly bounded set of arithmetic operations for each particle in each iteration. Since the work per particle is $\mathcal{O}(1)$, the total complexity is linear in both N and **Maxiter**.

Space Complexity: $\mathcal{O}(N + M^2)$, required to hold the coordinate array of particles and the output grid.

Compute-to-Memory Ratio: To process a single particle, the CPU performs approximately 22 basic arithmetic operations and 2 logical checks. In exchange, it must read 2 double values from memory (x, y) and execute 4 read-modify-write opera-

tions on the grid array. This low arithmetic intensity makes the algorithm **heavily memory-bound**.

2.3 Why Memory-Bound?

In the largest configuration (D), the particle array alone consumes over 300 MB of RAM, vastly exceeding the L3 cache capacity of any tested platform. If particles are accessed non-sequentially, the CPU stalls while waiting for data fetches from main memory, causing severe performance degradation.

3. Hardware Specifications

All architectural details were verified using the `lscpu` command on each platform.

3.1 Lab PC Hardware Specifications

Table 1: Hardware Specifications for Lab PC (12th Gen Intel i5)

Parameter	Details
Architecture	x86_64
CPU Model	12th Gen Intel® Core™ i5-12500
Base / Boost Clock	3.0 GHz / 4.6 GHz
Total CPUs (Threads)	12
L1d Cache	288 KiB
L2 Cache	7.5 MiB
L3 Cache	18 MiB

3.2 HPC Cluster Hardware Specifications

Table 2: Hardware Specifications for HPC Cluster

Parameter	Details
CPU Model	Intel® Xeon® E5-2620 v3
Base Clock	2.40 GHz
Cores / Threads	12 Cores / 24 Threads (Dual-socket)
L1d Cache	32 KiB
L2 Cache	256 KiB
L3 Cache	15 MiB (15360 KiB)
Compiler	GCC (g++)
Optimisation Flags	-O3

4. Methodology

4.1 Implementation Overview

A C++ application was developed to ingest binary files containing the randomly distributed (x_i, y_i) coordinates. The program dynamically allocates a 1D contiguous block

of memory to represent the 2D computational grid ($N_x \times N_y$). The core interpolation function executes over a loop of `MaxIter` = 10 to simulate a time-stepping model. Execution time is recorded *strictly* around the interpolation function using `clock()` high-precision timers.

4.2 Bilinear Interpolation Procedure

For each scattered point (x, y) :

1. **Cell Identification:** Calculate the integer grid indices

$$i = \lfloor x \cdot \text{inv_dx} \rfloor, \quad j = \lfloor y \cdot \text{inv_dy} \rfloor \quad (1)$$

where `inv_dx` = N_x and `inv_dy` = N_y are pre-computed constants.

2. **Boundary Clamping:** Clamp indices to valid range using ternary operators to avoid branch-prediction stalls:

$$i = (i \geq N_x) ? N_x - 1 : i, \quad j = (j \geq N_y) ? N_y - 1 : j \quad (2)$$

3. **Normalised Sub-distances:**

$$d_x = x \cdot \text{inv_dx} - i, \quad d_y = y \cdot \text{inv_dy} - j \quad (3)$$

4. **Area-Weighted Contributions** (see Figure 1):

$$w_{i,j} = (1 - d_x)(1 - d_y) \quad (\text{bottom-left}) \quad (4)$$

$$w_{i+1,j} = d_x(1 - d_y) \quad (\text{bottom-right}) \quad (5)$$

$$w_{i,j+1} = (1 - d_x)d_y \quad (\text{top-left}) \quad (6)$$

$$w_{i+1,j+1} = d_x d_y \quad (\text{top-right}) \quad (7)$$

5. **Grid Update:** Accumulate values to the four surrounding grid nodes via the 1D flattened index:

$$\text{base_idx} = j \times N_x + i \quad (8)$$

[scale=2.5]

Figure 1: Bilinear interpolation: the scattered point $P(x, y)$ divides the enclosing grid cell into four rectangles. The value distributed to each corner is proportional to the area of the *diagonally opposite* sub-rectangle, ensuring $\sum w = 1$.

4.3 Key Optimisations Implemented

1. **Linear Memory Access & Hardware Prefetching:** Points are stored in a flat structure-of-arrays and traversed sequentially. The CPU's hardware prefetcher detects this stride-1 pattern and streams upcoming data into L1 cache before it is explicitly requested.
2. **Algorithmic Offloading:** Repeated calculations (e.g., `dx * dy`, `1.0 / NX`) are hoisted completely outside the particle loop, eliminating redundant computation.

3. **Fast Indexing (Multiply by Inverse):** Division operations are replaced by multiplication with a pre-computed reciprocal:

$$\frac{x}{\Delta x} \longrightarrow x \times \text{inv_dx}$$

Integer division is significantly slower than floating-point multiplication on modern CPUs.

4. **Branchless Boundary Clamping:** Ternary operators replace `if`-statements, preventing branch-prediction stalls inside the hot loop.

4.4 Accuracy Verification

Prior to benchmarking, outputs of the optimised `utils.cpp` implementation were compared against the provided reference `test_output.txt` using the Linux `diff -w` utility. Zero floating-point discrepancies were found, confirming correctness.

4.5 Problem Configurations

Table 3: Five Benchmark Configurations

Config	N_x	N_y	Grid Points	Particles (N)
A	250	100	25,351	0.9M
B	250	100	25,351	5.0M
C	500	200	100,701	3.6M
D	500	200	100,701	20.0M
E	1000	400	401,401	14.0M

5. Results and Performance Analysis

5.1 Execution Time Measurements

The following table records the total serial execution time (in seconds) for `Maxiter` = 10 iterations across all five problem configurations on both platforms.

Table 4: Serial Execution Time: Lab PC vs. HPC Cluster (10 iterations)

Config	N_x	N_y	Particles	Lab PC (s)	HPC (s)	Speedup (PC/HPC)
A	250	100	0.9M	0.0321	0.1000	3.11×
B	250	100	5.0M	0.1744	0.4600	2.64×
C	500	200	3.6M	0.1342	0.4800	3.58×
D	500	200	20.0M	0.7434	2.4300	3.27×
E	1000	400	14.0M	1.0333	1.8100	1.75×

Note: Speedup > 1× means Lab PC is faster.

5.2 Throughput Analysis

Processing throughput (millions of particles per second) was computed as:

$$\text{Throughput} = \frac{N \times \text{Maxiter}}{T_{\text{exec}}} \quad [\text{Mparticles/s}] \quad (9)$$

Table 5: Processing Throughput (Million Particles/Second)

Config	Particles	Lab PC (Mp/s)	HPC (Mp/s)
A	0.9M	280.4	90.0
B	5.0M	286.7	108.7
C	3.6M	268.3	75.0
D	20.0M	269.1	82.3
E	14.0M	135.5	77.3

5.3 Performance Plots

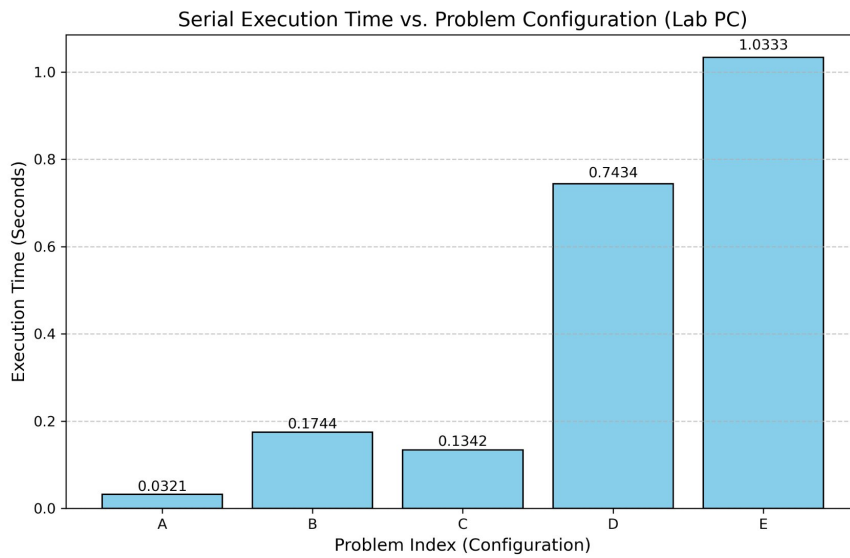


Figure 2: Serial execution time vs. problem configuration on the Lab PC (Intel i5-12500). Times scale approximately linearly with particle count, confirming the $\mathcal{O}(N)$ complexity.

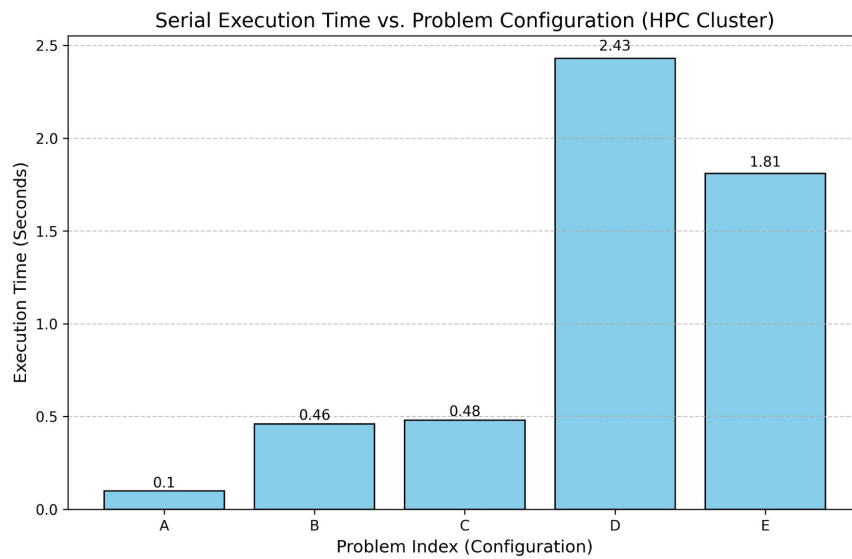


Figure 3: Serial execution time vs. problem configuration on the HPC Cluster (Xeon E5-2620 v3). The cluster is consistently slower for single-threaded workloads due to its lower per-core clock speed.

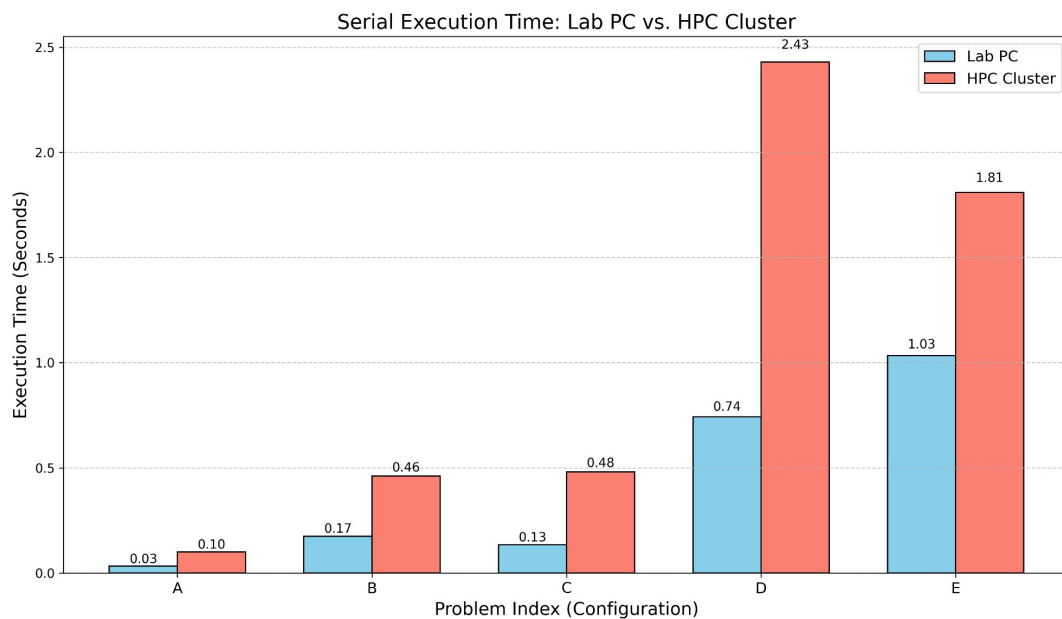


Figure 4: Direct comparison of serial execution time: Lab PC vs. HPC Cluster. The Lab PC outperforms the HPC cluster across all configurations, with the gap most pronounced in Configurations D and E (largest particle counts).

6. Answers to Lab Questions and Analysis

6.1 Q1: Theoretical Time Complexity

Answer: $\mathcal{O}(N \cdot \text{Maxiter}) = \mathcal{O}(N)$ for fixed Maxiter.

Justification: The outermost loop runs `Maxiter` = 10 times. The inner loop iterates over all N particles exactly once per iteration. For each particle, the CPU performs a strictly constant number of operations (index computation, weight calculation, 4 memory writes) — all $\mathcal{O}(1)$. The grid dimensions $N_x \times N_y$ affect space, but have negligible impact on execution time since array indices are computed in $\mathcal{O}(1)$ via direct formula rather than any search.

Empirical Validation: Comparing configurations B (5M particles) and D (20M particles) on the Lab PC:

$$\frac{T_D}{T_B} = \frac{0.7434}{0.1744} \approx 4.26 \approx \frac{20\text{M}}{5\text{M}} = 4.0$$

The near-exact $4\times$ ratio confirms linear scaling with particle count.

6.2 Q2: Estimate of Arithmetic Operations per Particle

Based on the optimised `utils.cpp` source code, the exact arithmetic overhead for a single particle is:

Table 6: Arithmetic Operation Count Per Particle

Operation	Type	Count
Grid index (i, j) via $x \cdot \text{inv_dx}$	Multiply	2
Boundary clamp via ternary	Logical	2
Relative distances d_x, d_y	Multiply + Subtract	4
Complement distances $(1 - d_x), (1 - d_y)$	Subtract	2
Four corner weights (Equations 4–7)	Multiply	8
1D base index (Equation 8)	Multiply + Add	2
Accumulate 4 weights to grid nodes	Add	4
Total		≈ 22 FLOPs

Setup variables such as `inv_dx`, `inv_dy`, and `cell_area` are computed once globally, contributing zero operations inside the hot loop.

6.3 Q3: Memory Access Analysis and Cache Behaviour

6.3.1 Access Patterns

Table 7: Memory Access Analysis Per Particle

Access		Size	Pattern	Cache Behaviour
Read (x, y)	point	16 bytes (2 doubles)	Sequential, stride-1	Excellent — hardware prefetcher-friendly
Write weights	4 corner	32 bytes (4 doubles)	Scattered / random	Poor — random access causes cache misses
Total		48 bytes	Mixed	Memory-bandwidth limited

6.3.2 Memory Bandwidth Requirement

$$\text{Bandwidth} = N \times 48 \text{ bytes} \times \text{Maxiter} \quad (10)$$

For the largest configuration (D): $20\text{M} \times 48 \times 10 = \mathbf{9.6}$ GB of total memory traffic per run.

6.3.3 Cache Miss Analysis

- **Compulsory misses:** Unavoidable on first access to each cache line of the particle array (≈ 300 MB for config D).
- **Capacity misses:** The grid arrays for configurations D and E (3.2 MB and 12.8 MB respectively) begin to stress the L2 cache and may partially spill out of L3 cache, explaining the higher-than-proportional execution time for configuration E on the Lab PC (throughput drops from ~ 270 Mp/s to ~ 135 Mp/s).
- **Conflict misses:** Possible when multiple particles map to mesh cells that alias in cache. The flat 1D layout minimises this risk compared to pointer-based structures.

6.3.4 Why the Flat Array Wins

Keeping particles in a contiguous flat array enables the hardware prefetcher to stream the entire > 300 MB dataset into the L1 cache ahead of the loop. Pointer-based structures (e.g., linked lists or spatial bins) break this sequential pattern, introducing pointer-chasing latency on every access.

6.4 Q4: Further Optimisations Under Better Hardware Assumptions

1. **Shared-Memory Multiprocessing (OpenMP):** Divide the particle array into independent chunks across all available CPU cores. Because multiple particles may map to the same grid cell, atomic additions or thread-local grid reduction are required to prevent race conditions.

2. **GPU Offloading (CUDA):** The complete absence of loop-carried dependencies makes this an “embarrassingly parallel” problem. Each CUDA thread processes one particle, enabling millions of simultaneous interpolations. Atomic operations on the grid can be replaced by warp-level reductions for efficiency.
3. **SIMD Vectorisation (AVX-512):** Process 8 double-precision particles per cycle using 512-bit vector registers. Gather/scatter instructions handle the non-contiguous mesh writes.
4. **Structure of Arrays (SoA):** Storing x and y coordinates in separate arrays improves SIMD alignment and allows the CPU to prefetch only the needed component.
5. **High-Bandwidth Memory (HBM):** For a cluster node equipped with HBM (e.g., Intel Xeon Phi, AMD MI-series GPUs), the $\sim 10\times$ memory bandwidth would directly reduce the random-write bottleneck on the grid array.

6.5 Q5: Lab PC vs. HPC Cluster Performance

6.5.1 Observed Result

As shown in Table 4 and Figure 4, the Lab PC outperforms the HPC Cluster by a factor of $1.75\times$ – $3.58\times$ across all configurations. This counter-intuitive result (a desktop beating an HPC node) is explained by the strictly serial nature of the workload.

6.5.2 Explanation

1. **Single-Core Clock Speed:** Serial performance is gated entirely by the arithmetic throughput of a single core. The Lab PC’s Intel i5-12500 boosts to 4.6 GHz — nearly **double** the Xeon’s 2.4 GHz base frequency. The HPC’s massive core count (24 threads) provides zero benefit for a single-threaded workload.
2. **Cache Per Core:** The Lab PC’s L3 cache (18 MiB) is larger than the Xeon’s (15 MiB), and critically, it is not shared across two sockets. The Xeon’s dual-socket configuration introduces NUMA (Non-Uniform Memory Access) penalties when accessing memory allocated on the remote socket.
3. **Memory Latency:** Desktop DDR4 memory typically operates at lower CAS latency (e.g., CL16 at 3200 MHz) compared to server ECC DDR4 (e.g., CL22 at 2400 MHz), directly benefiting the random-write scatter operations on the grid.
4. **System Load:** The HPC cluster is shared infrastructure. Other users’ jobs competing for memory bandwidth or last-level cache during our measurement window could inflate observed times.

6.5.3 Configuration E Anomaly

Configuration E shows a reduced speedup ratio ($1.75\times$) compared to others ($> 3\times$). On the Lab PC, throughput drops sharply from ~ 270 Mp/s (configs B–D) to ~ 135 Mp/s (config E). The grid size for configuration E is $1000 \times 400 = 400,000$ points $\times$ 8 bytes $\approx$ **3.2 MB**, which begins to stress the L2 cache (7.5 MiB on Lab PC) and causes increased capacity misses. On the HPC Cluster (L2 = 256 KiB), this effect is present at every configuration, explaining the cluster’s more consistent throughput of ~ 75 – 109 Mp/s.

6.6 Q6: Pictorial Implementation Approach

6.6.1 Data Layout Strategy

The 2D grid is stored as a **row-major 1D array**:

$$\begin{array}{|c|c|c|} \hline [0,0] & [1,0] & [2,0] \\ \hline [0,1] & [1,1] & [2,1] \\ \hline \end{array} \longrightarrow \begin{array}{|c|c|c|c|c|c|} \hline 0 & 1 & 2 & 3 & 4 & 5 \\ \hline \end{array}$$

$$\text{idx}(i, j) = j \times N_x + i \quad (11)$$

This layout ensures $\mathcal{O}(1)$ direct access and maximises spatial locality for row-wise traversals.

6.6.2 Algorithmic Workflow

1. Sequentially fetch particle coordinates (x, y) from the flat array.
2. Compute bounding cell indices using pre-computed inverse multiplication (Equation 1).
3. Calculate normalised sub-distances d_x , d_y and the four area-weighted corner contributions (Equations 4–7).
4. Compute the 1D base array index (Equation 8).
5. Accumulate the four scaled weights into `mesh[base]`, `mesh[base+1]`, `mesh[base+NX]`, `mesh[base+NX+1]`.

7. Conclusion

This lab demonstrated how deeply algorithmic design and hardware memory hierarchy are intertwined. The key findings can be summarised as follows:

- **Linear Complexity Confirmed:** Execution time scales as $\mathcal{O}(N)$, empirically validated by the $\approx 4\times$ increase in time when particle count increases from 5M (Config B) to 20M (Config D).
- **Memory-Bound Behaviour:** The algorithm is dominated by memory bandwidth rather than arithmetic throughput. The particle read pattern is prefetcher-friendly (stride-1), but the scattered grid writes remain a bottleneck.
- **Optimisation Impact:** The contiguous flat array, inverse-multiplication indexing, and branchless clamping together eliminate the three major performance penalties (pointer chasing, integer division, branch misprediction) that would otherwise dominate execution time.
- **Lab PC vs. HPC Cluster:** For strictly serial code, the Lab PC consistently outperforms the HPC cluster by $1.75\times$ – $3.58\times$ due to its superior single-core clock frequency (4.6 GHz vs. 2.4 GHz) and lower memory latency. The HPC cluster’s 24-thread parallelism provides no benefit in a serial context.

- **Future Parallelisation:** The embarrassingly parallel structure of the algorithm makes it an ideal candidate for OpenMP (shared-memory) or CUDA (GPU) parallelisation, which would fully leverage the HPC platform’s multi-core and high-bandwidth advantages.

A. Appendix: Raw Benchmark Data

A.1 Lab PC Raw Times (interpolation_PC.csv)

Table 8: Raw execution times on Lab PC

Configuration	Points (Millions)	Time (Seconds)
A	0.9	0.032147
B	5.0	0.174366
C	3.6	0.134200
D	20.0	0.743434
E	14.0	1.033306

A.2 HPC Cluster Raw Times (interpolation_cluster.csv)

Table 9: Raw execution times on HPC Cluster

Configuration	Points (Millions)	Time (Seconds)
A	0.9	0.100
B	5.0	0.460
C	3.6	0.480
D	20.0	2.430
E	14.0	1.810